

```
❯ ./run_analysis --target="Forja del Servidor Central"
```

Análisis Exhaustivo: Arquitectura de Streaming y Reproducción Concurrente en Android

Ingeniería de Mando para el Servidor Central de Eregion

Gemini Notebook

Contexto Táctico

> _

Celebrimbor requiere una infraestructura informática de mando definitiva para coordinar las defensas de Eregion mediante una consola interactiva avanzada.

>



Selección Dinámica: Componente Spinner para cambiar de puesto en tiempo real.



Streaming Remoto: Motor ExoPlayer para sintonizar URLs de vídeo .mp4.



Monitoreo Automático: Intercepción de eventos de red y buffer sin bloquear la UI.



Alerta Concurrente: Motor nativo MediaPlayer para disparo acústico paralelo.

Gemini Notebook

Transformación de Código a Wireframe

```xml
<LinearLayout xmlns:android="http://www.androidapi.com/xmpl..."
    xmlns:app="http://www.google.com/androidey-app"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- Cabecera: selector desplegable de canales -->
    <Spinner
        android:id="@+id/spinnerCanales"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:minHeight="48dp" />

    <!-- Zona central: aquí se pinta el vídeo -->
    <androidx.media3.ui.PlayerView
        android:id="@+id/playerView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        app:resize_mode="fit" />

    <!-- Zona inferior: texto de estado -->
    <TextView
        android:id="@+id/textoEstado"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:textStyle="bold" />

</LinearLayout>
```



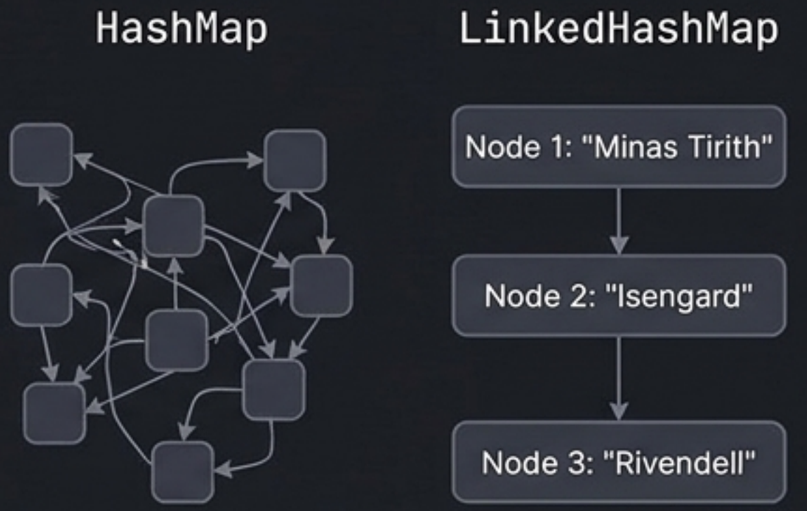
Gemini Notebook

```
// Las vistas del layout. Las dejamos "lateinit" porque no existen hasta
// que se infla el XML en onCreate, pero nunca se usan antes de eso.
private lateinit var spinnerCanales: Spinner
private lateinit var playerView: PlayerView
private lateinit var textoEstado: TextView

// Motor de vídeo (ExoPlayer) y motor de la alarma (MediaPlayer).
// Los dos son nullable a propósito: el de vídeo lo creamos al arrancar y
// lo ponemos a null al liberarlo; el de la alarma solo existe mientras suena.
private var exoPlayer: ExoPlayer? = null
private var alarma: MediaPlayer? = null

/**
 * Mapa de canales: nombre que ve el operador -> URL del vídeo .mp4.
 * Usamos LinkedHashMap para que el desplegable salga siempre en el mismo
 * orden en que los escribimos (un HashMap normal no lo garantiza).
 *
 * OJO: estas URLs son de ejemplo (vídeos públicos de prueba). Hay que
 * sustituirlas por las que facilite la cátedra.
 */
private val canales: LinkedHashMap<String, String> = linkedMapOf(
    "Minas Tirith" to "https://storage.googleapis.com/gtv-videos-bucket/sample/BigBuckBunny.mp4",
    "Isengard" to "https://storage.googleapis.com/gtv-videos-bucket/sample/ElephantsDream.mp4",
    "Rivendell" to "https://storage.googleapis.com/gtv-videos-bucket/sample/ForBiggerBlazes.mp4"
)
```

Decisión de Diseño: Motores nullable por diseño para control estricto de memoria.



Garantiza orden determinista en la interfaz del operador.

Gemini Notebook

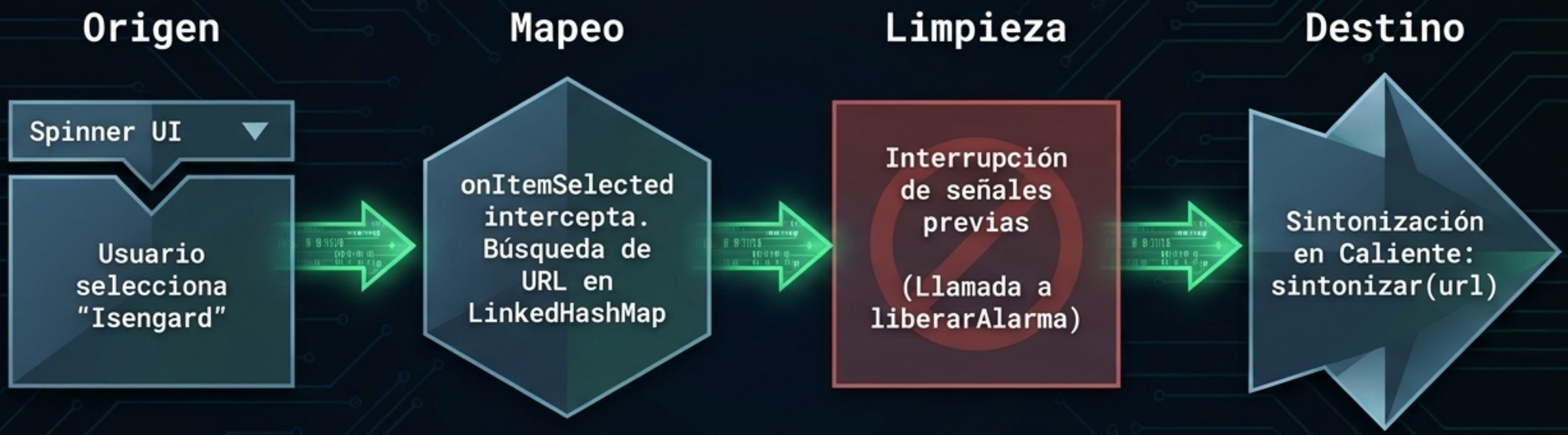
```kotlin
/**
 * Crea el ExoPlayer, lo conecta a la PlayerView y le cuelga el "oyente"
 * que se entera de todo lo que le pasa al buffer y al estado de la señal.
 */
private fun prepararReproductor() {
    val player = ExoPlayer.Builder( context = this).build()
    playerView.player = player
    player.addListener(oyenteDeEstado)
    exoPlayer = player
}
```

Wiring Diagram



Inyección del “chivato” que observará la red sin bloquear el hilo principal (Main Thread).

Gemini Notebook

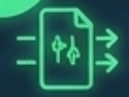


Inyección mediante ArrayAdapter. La selección detiene alertas residuales y enruta la nueva fuente de datos en caliente.

Gemini Notebook

```
private fun sintonizar(url: String) {
    exoPlayer?.let { player ->
        player.setMediaItem(MediaItem.fromUri(url))
        player.prepare()
        player.playWhenReady = true
    }
}
```

1



Inyección de la nueva fuente remota (.mp4).

2



Delegación del buffering y negociación de red al motor de ExoPlayer.

3



Instrucción de arranque asíncrono: el vídeo iniciará automáticamente en cuanto el búfer sea suficiente.

Gemini Notebook

```kotlin
/**
 * El "chivato" del reproductor. ExoPlayer llama a estos métodos solo
 * cuando algo cambia, así que no hace falta ir preguntándole nada.
 */
private val oyenteDeEstado = object : Player.Listener {  2 Usages

    override fun onPlaybackStateChanged(playbackState: Int) {
        when (playbackState) {
            // Está descargando datos y todavía no puede reproducir.
            Player.STATE_BUFFERING ->
                textoEstado.setText(R.string.estado_cargando)

            // Ya tiene suficiente búfer: se ve el vídeo.
            Player.STATE_READY ->
                textoEstado.setText(R.string.estado_en_vivo)

            // El vídeo se acabó: avisamos en texto Y con sonido a la vez.
            Player.STATE_ENDED -> {
                textoEstado.setText(R.string.estado_terminada)
                dispararAlarma()
            }
        }
    }
```



Gemini Notebook

The Dual Engine Matrix: Ejecución Concurrente

	ExoPlayer (Motor Primario)	MediaPlayer (Motor Secundario)
Propósito	Streaming Remoto Complejo	Disparo Acústico (Alarma)
Fuente de Datos	URLs de red variables (GCP Bucket)	Recurso local físico (R.raw.campanas_alarma)
Binding de Interfaz	Requiere anclaje a vista gráfica (PlayerView)	'Headless' (Invisible, ejecución en segundo plano)
Complejidad de Ciclo	Alta (Gestión automática de búfer y códecs)	Fire-and-forget (Levanta, reproduce y se autodestruye)

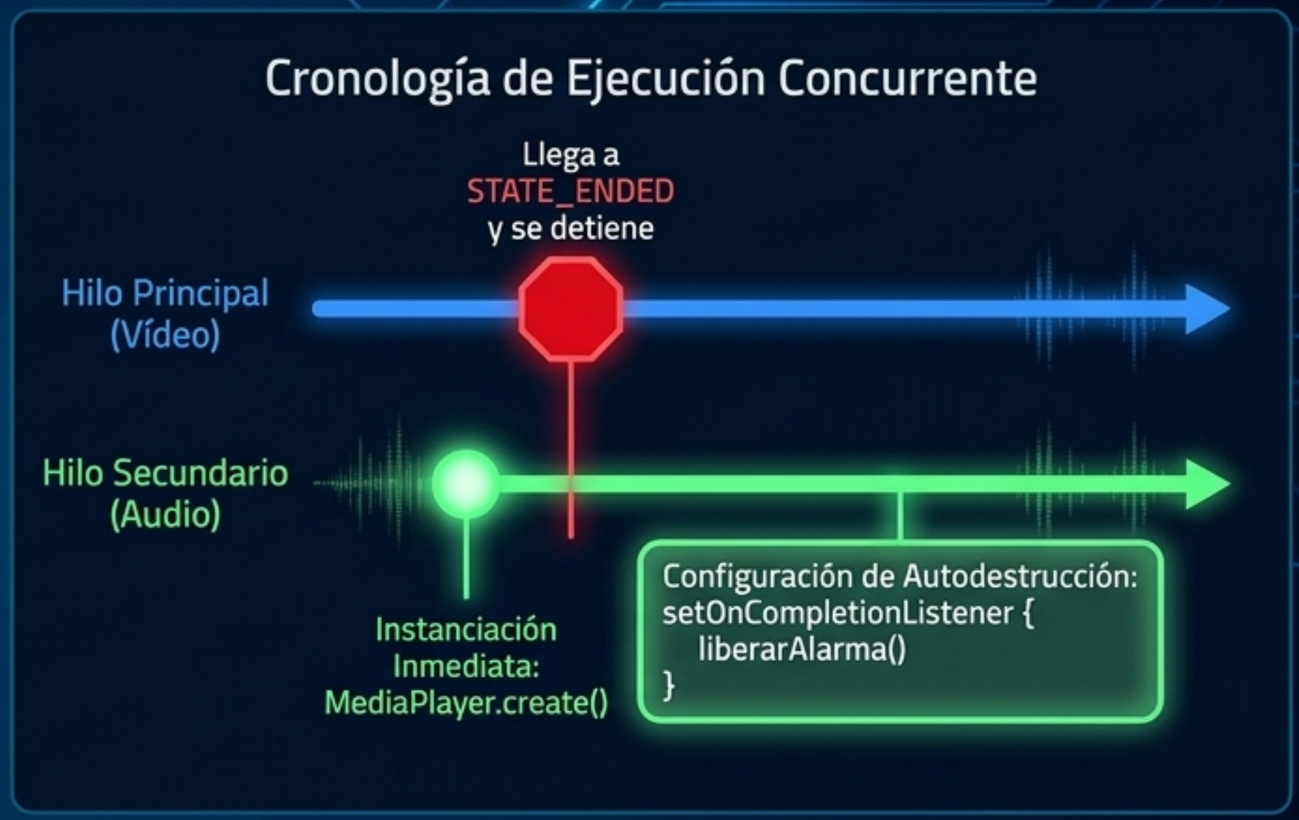
¿Por qué dos motores? Para levantar el audio de forma nativa e independiente sin interrumpir el hilo del motor principal.

Gemini Notebook

```kotlin
private fun dispararAlarma() {
    // El audio se lanza en su propio hilo.
    val mediaPlayer = MediaPlayer.create(this, R.raw.campanas_alarma)

    // Cuando termine el sonido, se libera la RAM.
    mediaPlayer.setOnCompletionListener { mp ->
        liberarAlarma(mp)
    }

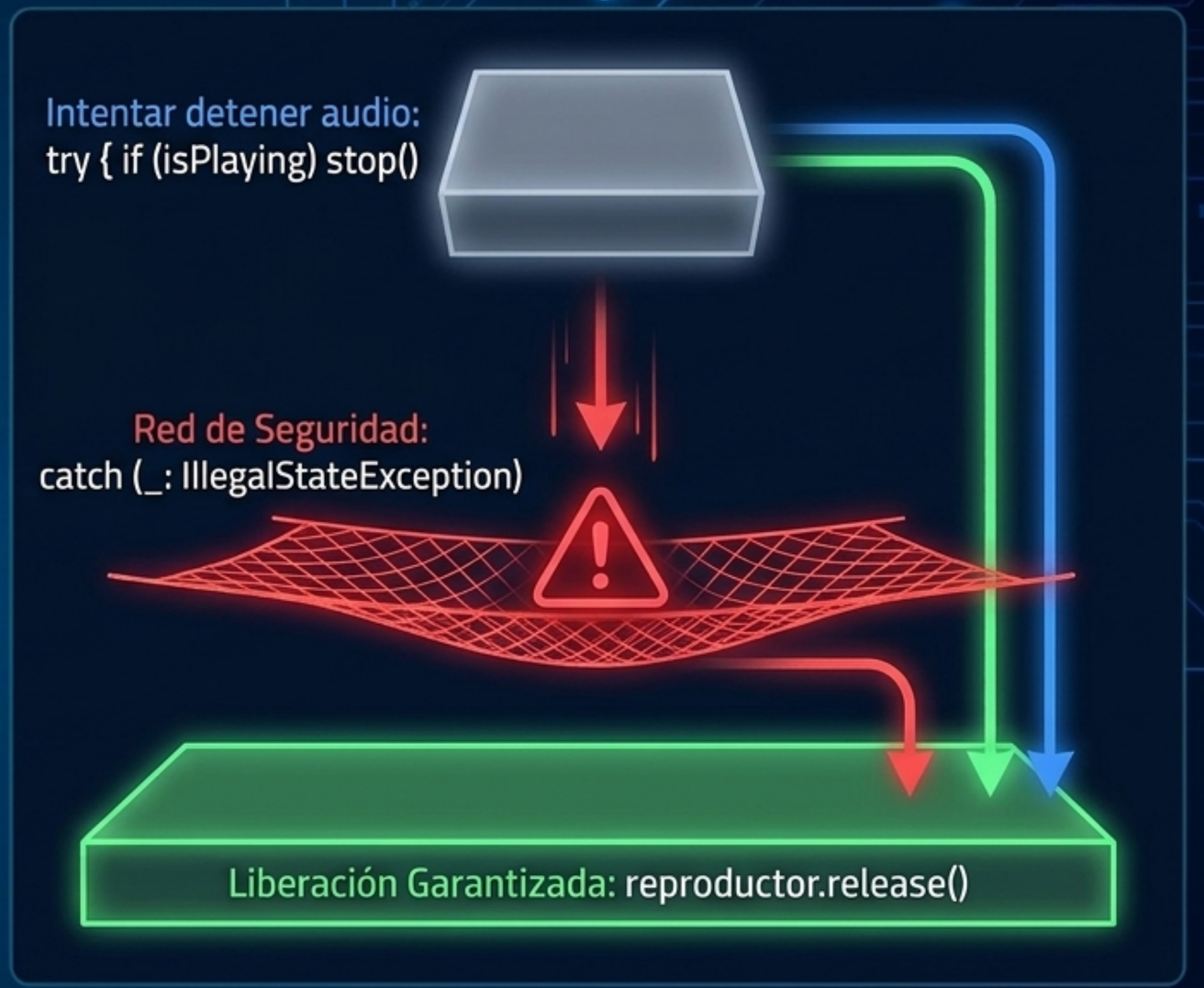
    mediaPlayer.start()
}
```



Ejecución Concurrente. El archivo estático de campanas se ejecuta en paralelo. La alarma libera sus recursos de RAM inmediatamente tras dejar de sonar.

Gemini Notebook

```kotlin
private fun liberarAlarma(mp: MediaPlayer?) {
    if (mp != null) {
        try {
            if (mp.isPlaying) {
                mp.stop()
            }
        } catch (_: IllegalStateException) {
            // Captura silenciosa
        }
        mp.release()
    }
}
```



Intercepción preventiva. Garantizamos la liberación física del hardware de audio sin importar el estado inestable del reproductor.

Gemini Notebook

```kotlin
override fun onStop() {
    super.onStop()
    exoPlayer?.pause()
}

override fun onDestroy() {
    super.onDestroy()
    oyenteDeEstado?.let { exoPlayer?.removeListener(it) }
    exoPlayer?.release()
    playerView = null
    liberarAlarma()
}
```

Ciclo de Vida de la Actividad y Limpieza de Memoria

onStop()

Pausa en 2º plano:
exoPlayer?.pause()

Ahorro inmediato de ancho de banda y batería si la consola se minimiza.

onDestroy()

1. Desvinculación de oyenteDeEstado (Evita memory leaks)
2. Liberación de códecs de vídeo (release)
3. Pérdida de referencias visuales (playerView = null)
4. Exterminio del motor de audio (liberarAlarma)

Cierre Limpio. Respetando la memoria de Eregion devolviendo todos los recursos de hardware al sistema.

Gemini Notebook



El Puente hacia el Exterior

Sin esta declaración de infraestructura en el AndroidManifest.xml,
la sofistiaada arquitectura interna sería bloqueada.
Permiso indispensable para acceder al Storage remoto (GCP).

Gemini Notebook

Master Flowchart



La Forja en Acción: Un flujo limpio, concurrente y libre de fugas de memoria. El sistema central está completamente operativo.

Gemini Notebook